Software Design of the EduAirControl Platform

María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas

Technology Program in Software Analysis and Development – 3145556, Servicio Nacional de Aprendizaje (SENA)

Software Analysis and Development – 3145556

All instructors of the program

October 15, 2026 · 2025-2026

Abstract

This document presents the software design of EduAirControl, an environmental monitoring and analysis platform for educational environments built as a microservices system. The design is organized in four views: graphical interfaces, architectural model, design patterns and data model. The interface view documents the design system (color, typography, spacing and component tokens), the inventory of twelve screens and the desktop and mobile mockup captures, together with the navigation maps and the role-based access matrix. The architectural view describes the microservices style with an API Gateway and event-driven communication, the service catalog with its ports, the synchronous (REST) and asynchronous (RabbitMQ with outbox and inbox) communication flows, and the security strategy based on RS256 JWT and JWKS. The patterns view traces each adopted pattern —hexagonal ports and adapters, repository, API Gateway, BFF, transactional outbox, idempotent inbox, CQRS read model, documented circuit breaker— to its concrete implementation in the code. The data view summarizes the six PostgreSQL schemas, their main tables and the modeling rules. The design was built on the project's executable source and governance documentation; where they disagree, the drift is documented. The document serves as a reference for implementation, testing and operation.

Keywords: software design, microservices architecture, design patterns, graphical interfaces, data model, hexagonal.

Table of Contents

Abstract 2

1. Introduction 4

2. Graphical Interfaces 5

    2.1 Design system 5

    2.2 Screen inventory 7

    2.3 Main screens 8

    2.4 Navigation and access control 19

3. Architectural Model 20

    3.1 Adopted architectural style 20

    3.2 Context and container views 20

    3.3 Service catalog 21

    3.4 Inter-service communication 22

    3.5 Security 23

    3.6 Internal architecture of the services 23

4. Design Patterns 24

    4.1 Architectural and microservices patterns 24

    4.2 GoF patterns present in the code 25

    4.3 Pattern traceability in the package structure 26

5. Data Model 27

    5.1 Modeling principles 27

    5.2 Schemas and main tables 27

    5.3 Business rules of the model 28

6. Implementation Diagrams 29

7. Conclusions 29

References 30

Software Design of the EduAirControl Platform

1. Introduction

This document is the software design of the EduAirControl project, within the Software Analysis and Development (ADSO) program. Its purpose is to specify how the system is built: its visual language and screens, its architectural structure, the patterns that organize the code and its data model. The design turns the decisions of the analysis (Deliverable 03) and the technical proposal (Deliverable 04) into a description verifiable against the implemented code.

The document was written taking as primary source the project's executable repositories —`apps/web` and `apps/mobile` frontends, the six Spring Boot microservices, the API Gateway and the infrastructure stack— and as secondary source the governance documentation (`docs-ea-control`), the API contracts and the database script (Deliverable 06). Where documentation and code disagree, the code's behavior is described and the drift is explicitly flagged.

The audience is the development team, the evaluating instructor and the testing and operations teams. The document is organized as follows: Section 2 (graphical interfaces), Section 3 (architectural model), Section 4 (design patterns), Section 5 (data model), Section 6 (implementation diagrams), Section 7 (conclusions) and references.

Table 1

Related deliverables

Deliverable	Content	Use in this document
`01_Documento_General_Proyecto`	Vision, scope and project status	General context
`02_Especificacion_Requisitos`	Requirements `RFx.y`, `RNFx` and use cases	Traceability of screens and services
`03_Analisis_Software`	Analysis, dynamic views and classes	Direct input for this design
`04_Propuesta_Tecnica`	Technology decisions	Rationale for the architectural model

Deliverable	Content	Use in this document
`06_Base_Datos`	Entity-relationship model and SQL script	Section 5 (data model)
`07_Pruebas` to `15_Enlace_Repositorio_GitHub`	Tests, quality, manuals and annexes	Downstream consumers of the design

2. Graphical Interfaces

The interface view documents EduAirControl's visual language and its screens. The source of truth is the frontend code (`ea-control-frontend`), from which the real design tokens were extracted; the high-fidelity mockups replicate those tokens for layout validation.

2.1 Design system

The design system defines the visual variables shared by the web and mobile applications. Tokens live in CSS custom properties (`apps/web/src/index.css`) and are mirrored one-to-one on mobile (`tokens.js`). The main reusable component family is the `.ds-*` classes in `design-system.css`.

Table 2

Main design tokens

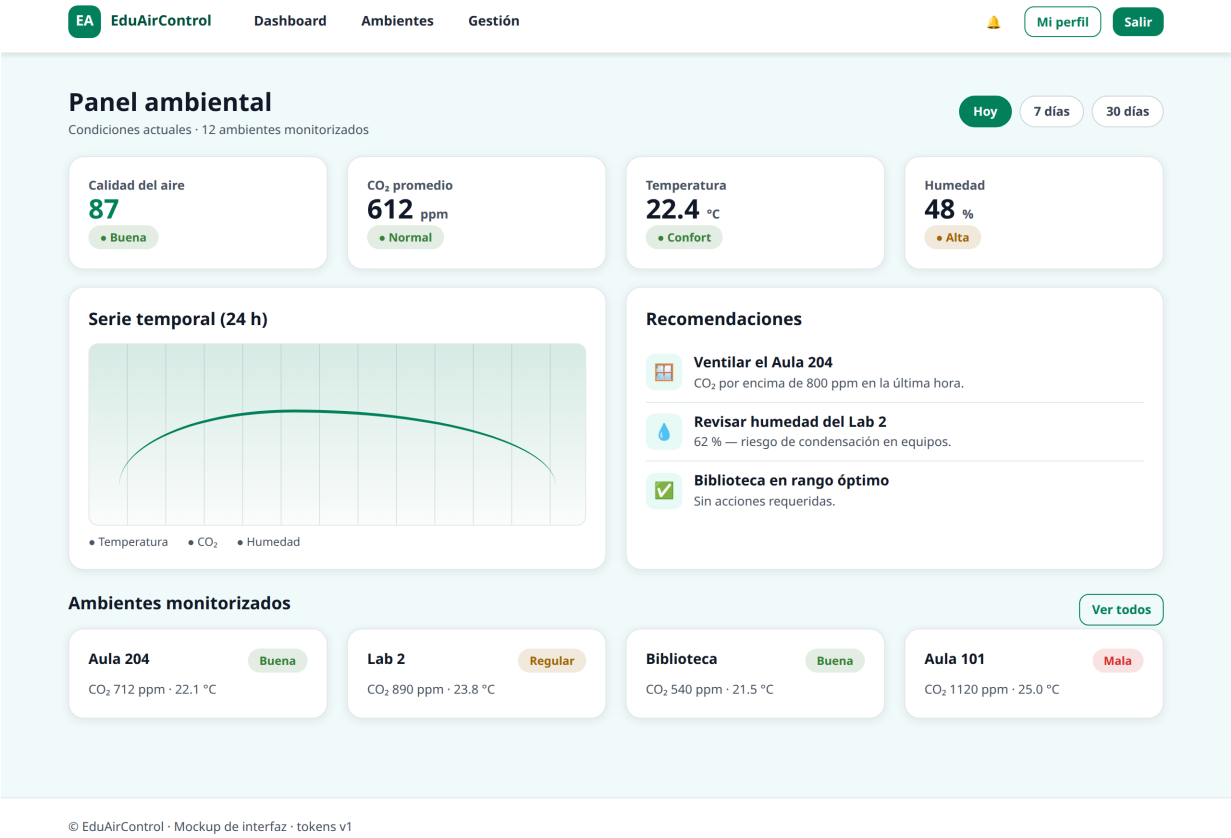
Category	Token	Value (light theme)	Use
Accent	`--accent`	`#01805b`	Primary buttons, links, active navigation
Accent	`--accent-light`	`#e8faf5`	Soft tinted surfaces
Semantic	`--color-success`	`#368139`	Confirmations, good status
Semantic	`--color-warning`	`#9d6602`	Warnings, regular status
Semantic	`--color-danger`	`#d63030`	Errors, bad status
Text	`--text-primary`	`#111827`	Primary text
Surface	`--bg-body` / `--bg-card`	`#f0fafa` / `#ffffff`	Page / card backgrounds

Category	Token	Value (light theme)	Use
Spacing	`--space-xs` … `--space-xl`	6 · 12 · 20 · 32 · 48 px	Layout rhythm
Radius	`--radius-sm` … `--radius-full`	10 · 16 · 20 · 28 · 999 px	Card borders and pills
Layers	`--z-panel` … `--z-loading`	100 · 200 · 300 · 400 · 500	Stacking order

Dark mode is a first-class theme (`body.dark-mode`), not a color inversion: it redefines backgrounds, text, borders and shadows. Alongside the three color-accessibility themes (protanopia `#007aa7`, deuteranopia `#4f7e00`, tritanopia `#5b4b8a`), the system supports text scaling (`--a11y-font-size` of 16, 18 and 20 px) and honors `prefers-color-scheme` when the user has not chosen a theme.

Figure 1

*Environmental dashboard screen (desktop).*Desktop mockup capture with the design-system tokens: metric cards, time series, recommendations and environment list.**



2.2 Screen inventory

The system has twelve functional screens. Eleven exist on web and eleven on mobile; two screens are platform-specific: the landing page is web-only and sensor provisioning is mobile-only.

Table 3

Screen inventory and navigation

#	Screen	Web	Mobile	Route / navigation	Minimum role
01	Landing Page	✅	—	`/landing`	Public

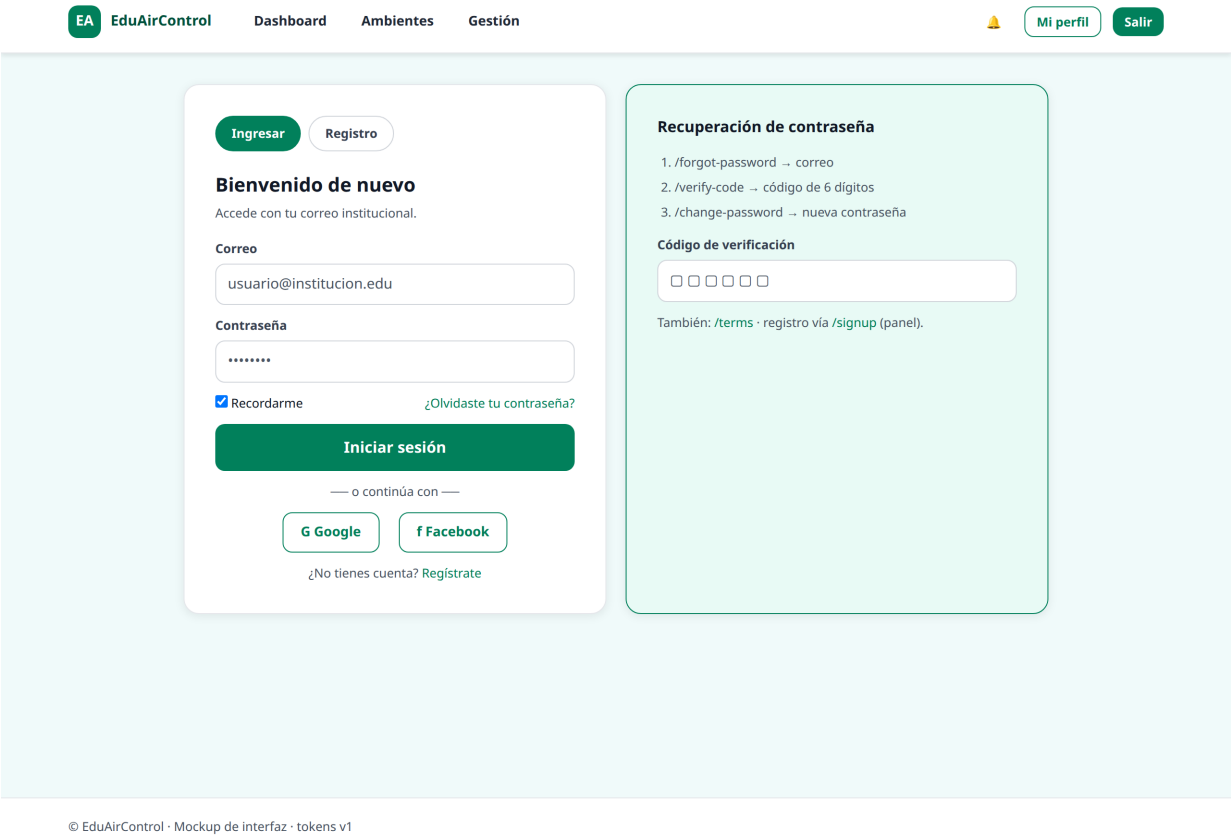
#	Screen	Web	Mobile	Route / navigation	Minimum role
02	Authentication	✅	✅	`/login` (+ recovery)	Guest
03	Product Guide	✅	✅	`/guide`	Public
04	Environmental Dashboard	✅	✅	`/dashboard`	USER
05	Educational Environments	✅	✅	`/all-environments`, `/environment/:id`	USER
06	Saved Environments	✅	✅	`/favorites`	USER
07	Environment Administration	✅	✅	`/management`	ADMIN
08	Super Admin Panel	✅	✅	`/admin/institutions`	SUPER_ADMIN
09	Alert Notifications	side panel	screen	overlay / `NotificationsPanel`	USER
10	User Profile	✅	✅	`/profile`	USER
11	User Settings	✅	✅	`/settings`	USER
12	Device Provisioning (BLE)	—	✅	`Provisioning` stack	ADMIN

2.3 Main screens

Authentication concentrates sign-in, registration and password recovery (four chained screens: `/forgot-password`, `/verify-code`, `/change-password`) and includes social login with Google and Facebook via a one-time code flow.

Figure 2

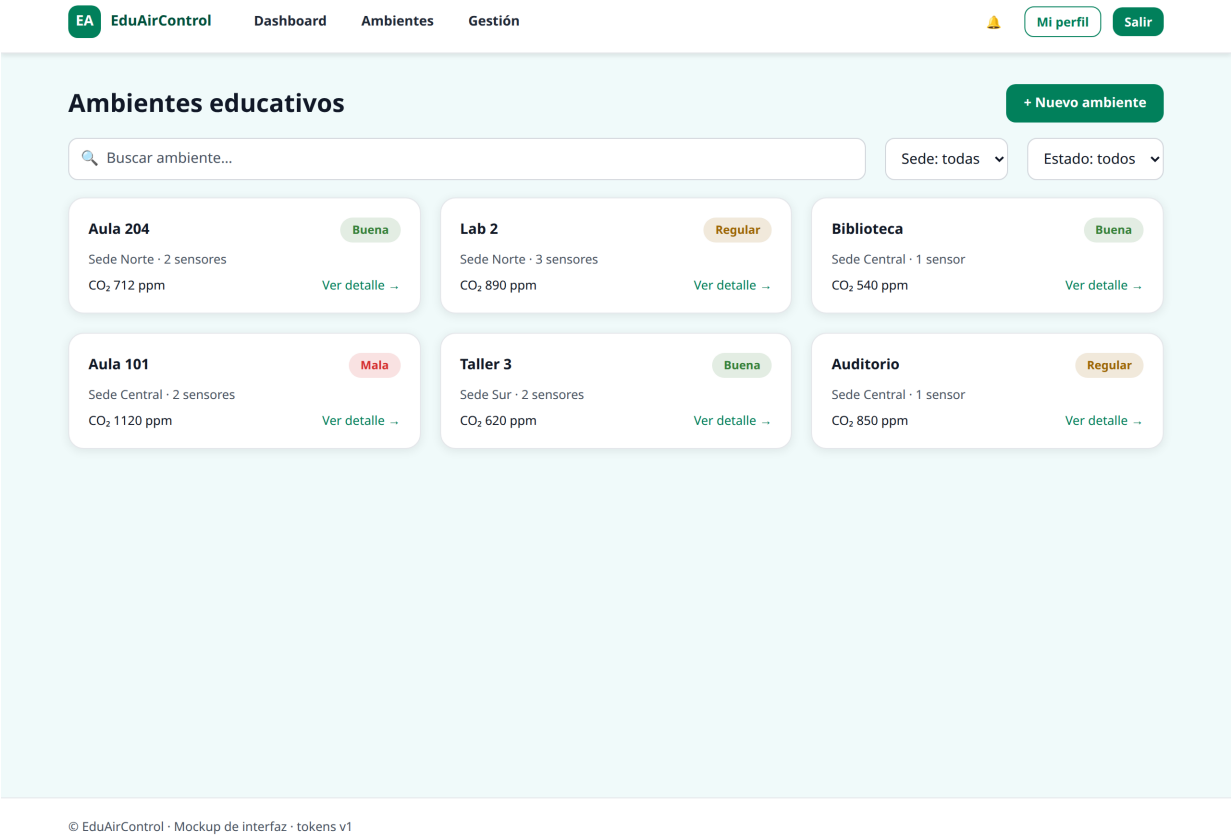
*Authentication screen (desktop).*Sign-in and registration panels, social login and the password recovery flow.**



The educational environments list reuses the `EnvironmentCard` component in three different contexts (dashboard, list and favorites) and applies filters by campus and status.

Figure 3

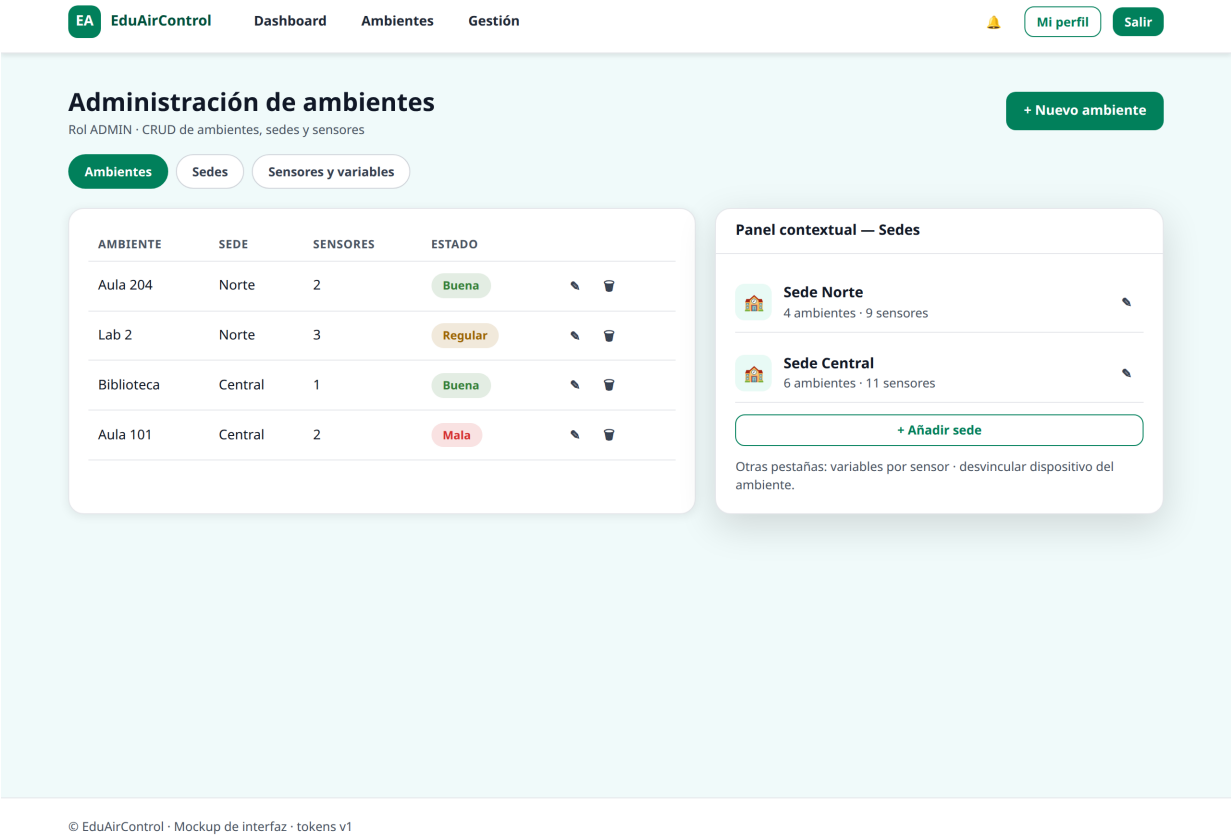
*Educational environments list (desktop).*Reusable cards with quality status, search filters and navigation to detail.**



Environment administration (ADMIN role) concentrates the environment CRUD, campus management, the sensor-variable panel and device unlinking. On mobile it adds the ESP32 BLE provisioning.

Figure 4

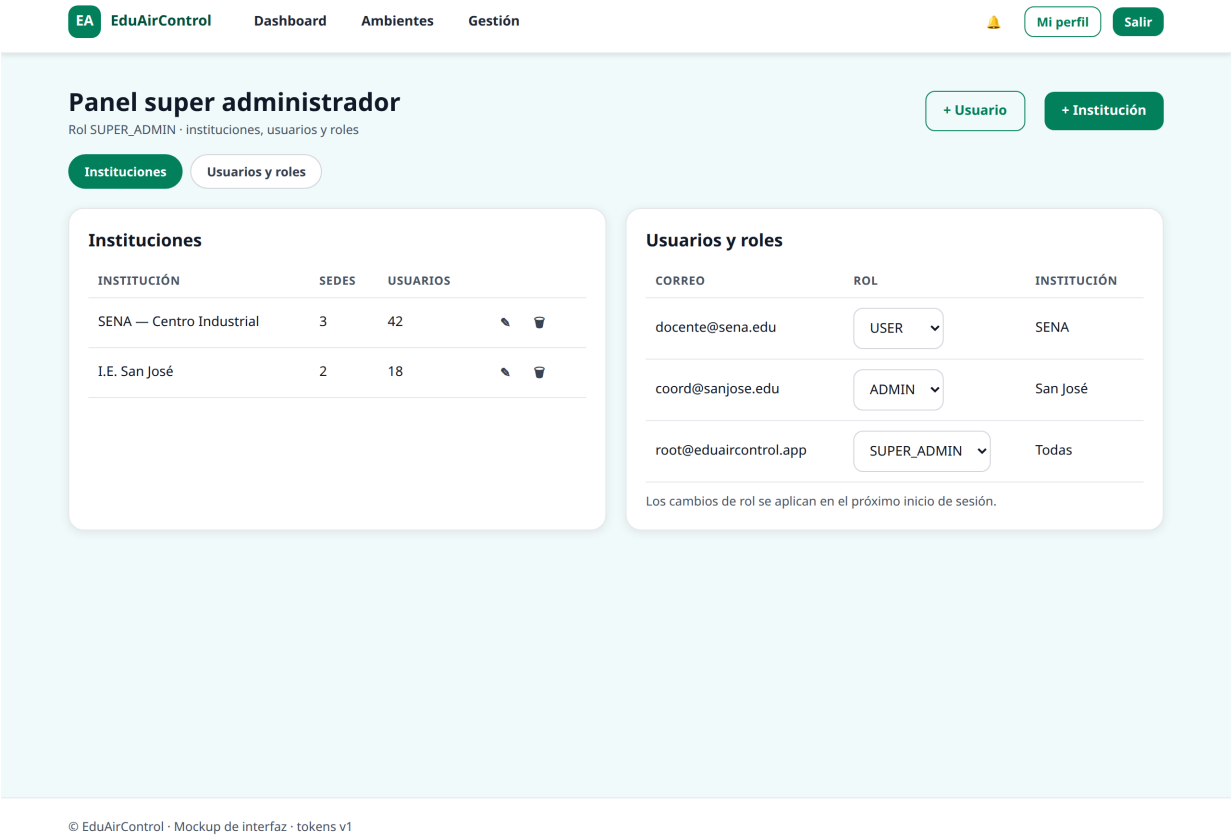
*Environment administration (desktop).*Environment table with edit and delete actions, and the campus side panel.**



The super admin panel (SUPER_ADMIN role) integrates institution management and user role assignment in a single screen.

Figure 5

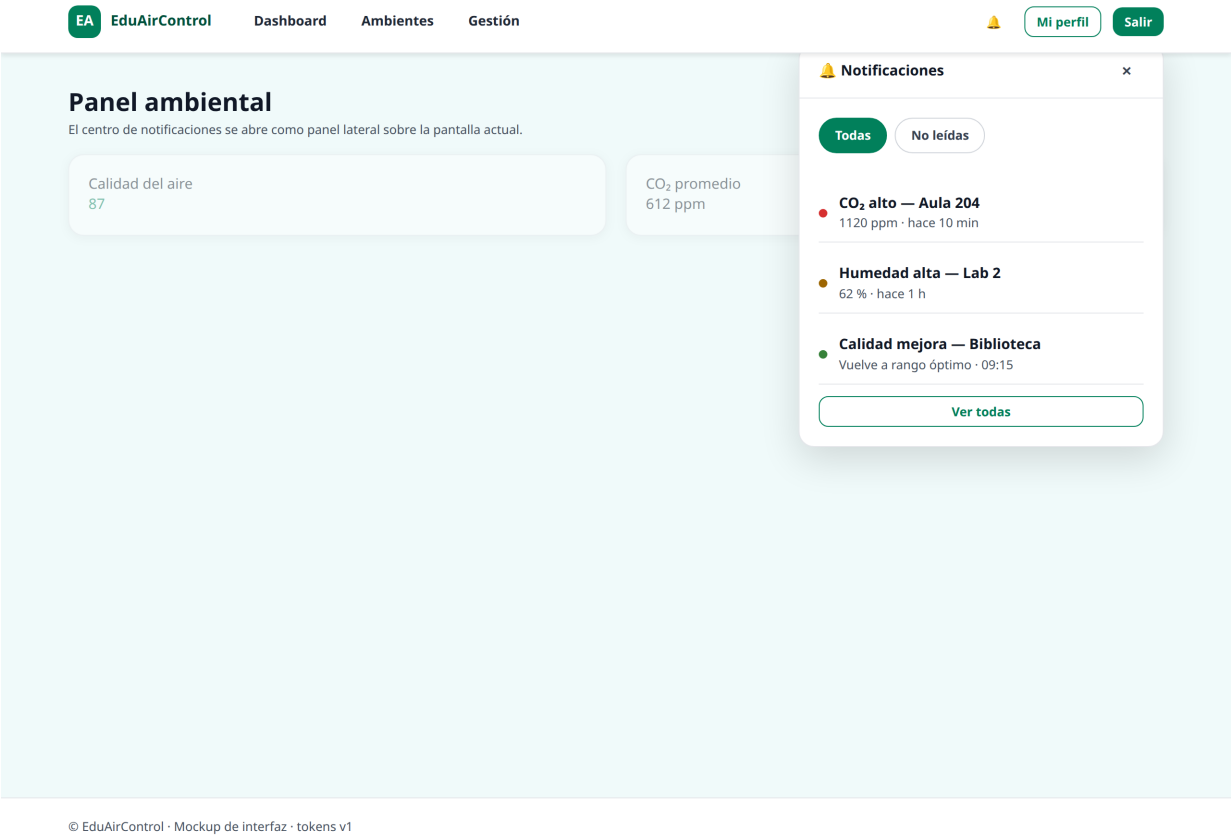
*Super admin panel (desktop).*Institution management and users with USER, ADMIN and SUPER_ADMIN role assignment.**



Notifications appear as a side overlay panel on web and as a screen on mobile: the same feature with two different navigation patterns, a deliberate decision driven by the available space on each platform.

Figure 6

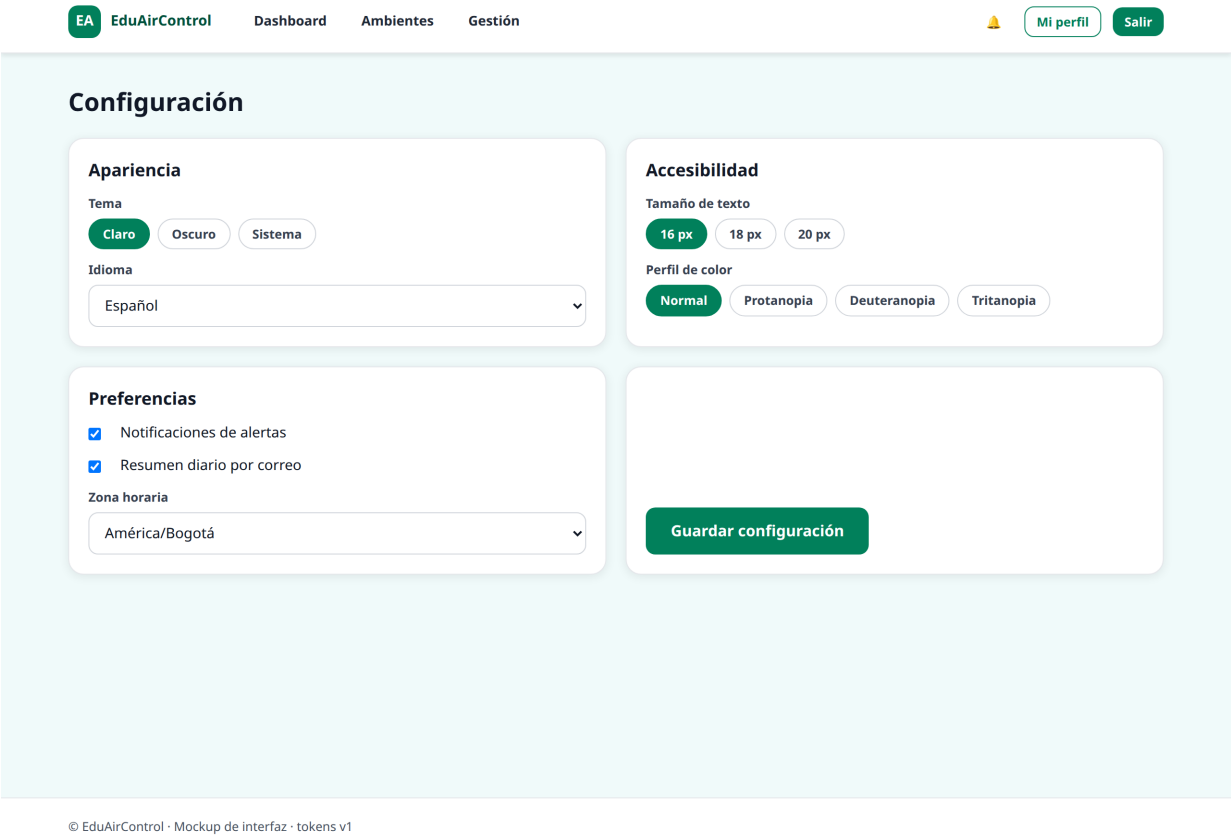
*Notification center as a side panel (desktop).*Alerts by severity with navigation to the originating environment.**



Settings groups appearance (light/dark/system theme), language (Spanish, English, Portuguese, French) and accessibility (text size and color profile).

Figure 7

*User settings (desktop).*Theme, language and accessibility options driven by the design-system tokens.**



On the mobile app, the dashboard summarizes environment metrics and BLE provisioning guides ESP32 pairing in three steps: scanning, network configuration and environment attachment.

Figure 8

Environmental dashboard (mobile).

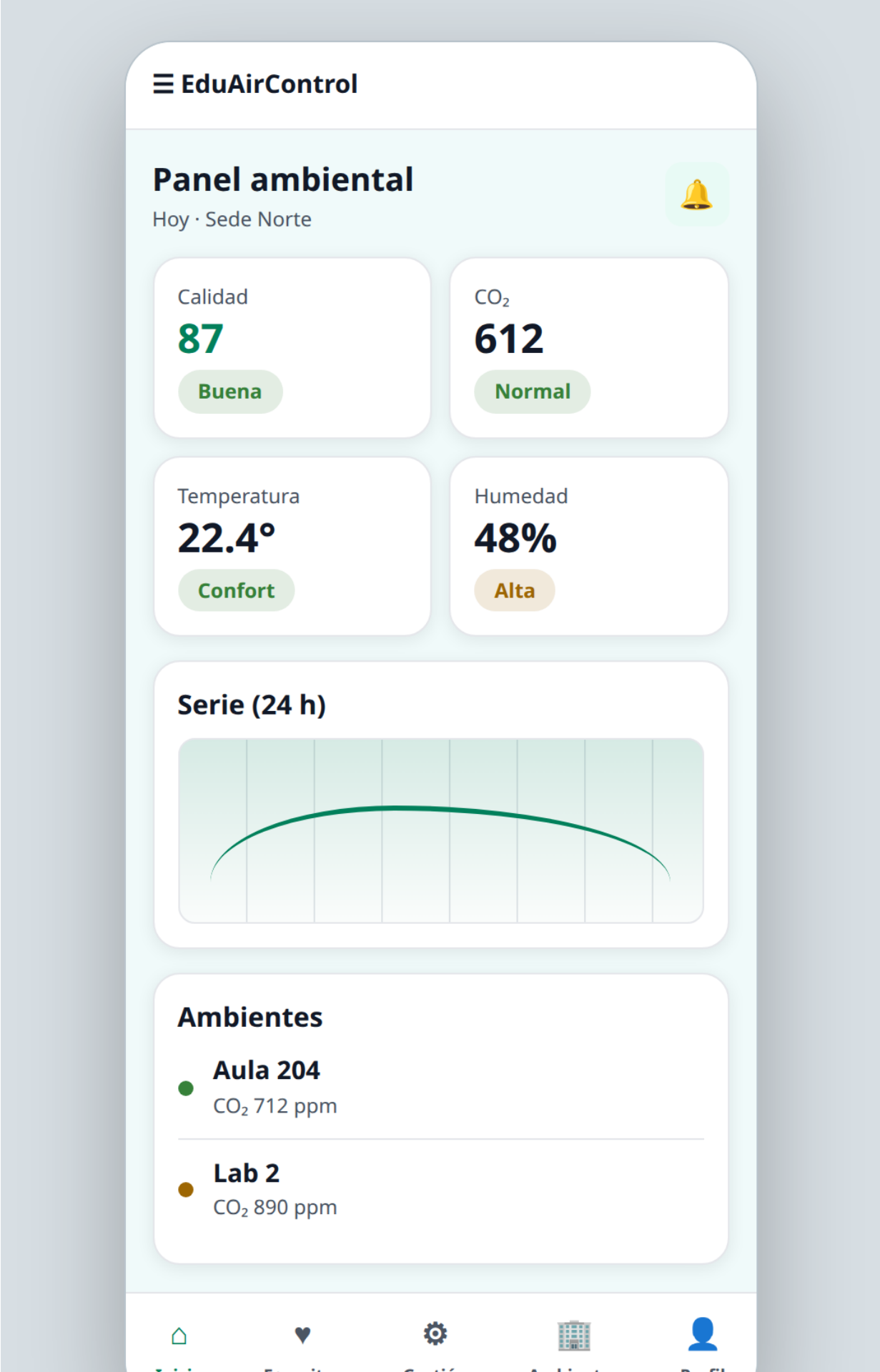
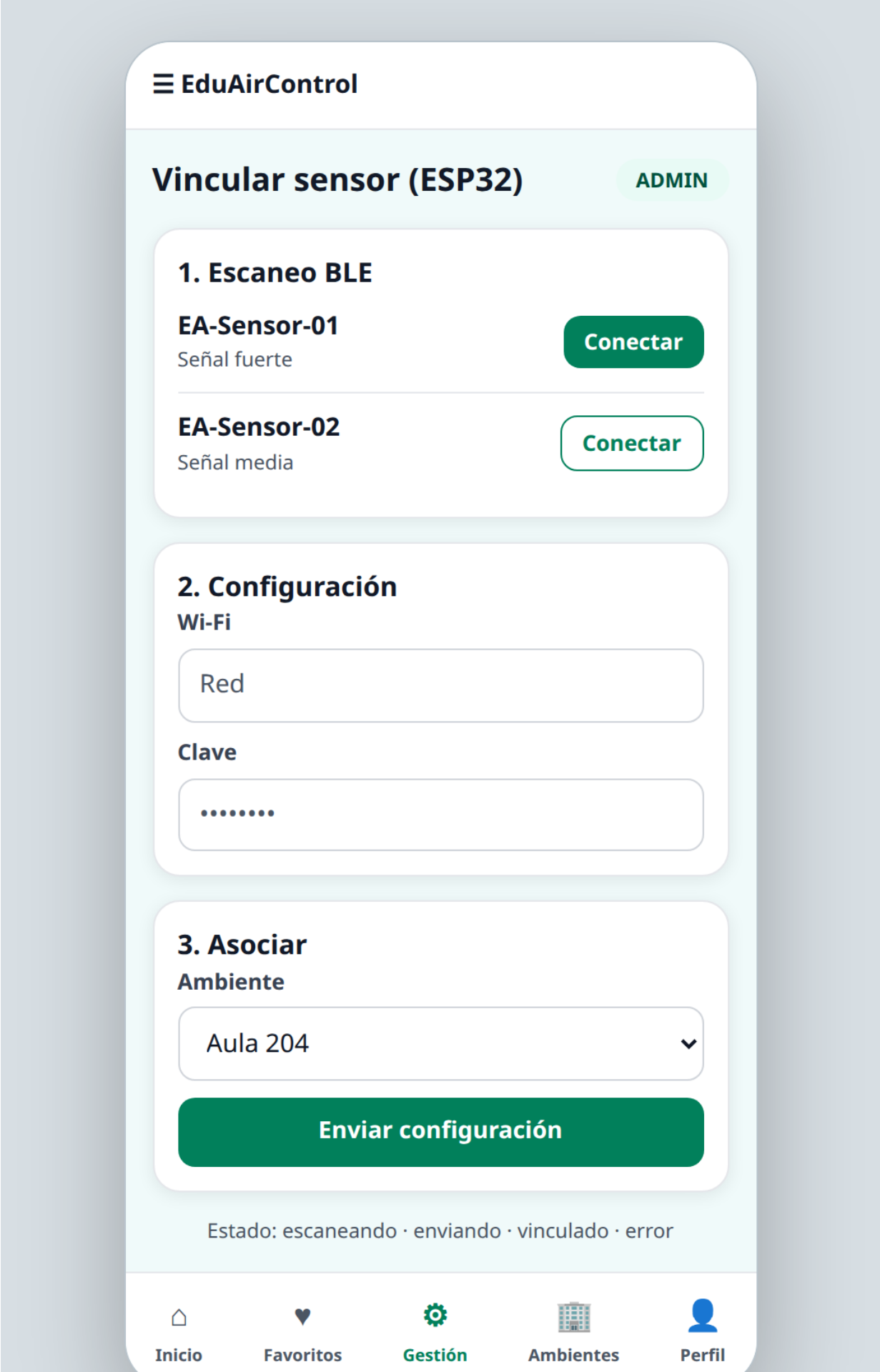

☰ EduAirControl
Panel ambiental
Hoy · Sede Norte
Calidad
87
Buena
CO₂
612
Normal
Temperatura
22.4°
Confort
Humedad
48%
Alta
Serie (24 h)
Ambientes
Aula 204
CO₂ 712 ppm
Lab 2
CO₂ 890 ppm

Figure 9

ESP32 sensor BLE provisioning (mobile).


☰ EduAirControl
Vincular sensor (ESP32)
ADMIN
1. Escaneo BLE
EA-Sensor-01
Señal fuerte
Conectar
EA-Sensor-02
Señal media
Conectar
2. Configuración
Wi-Fi
Red
Clave
••••••••
3. Asociar
Ambiente
Aula 204
Enviar configuración
Estado: escaneando · enviando · vinculado · error
Inicio
Favoritos
Gestión
Ambientes
Perfil

2.4 Navigation and access control

Web navigation is defined in `App.jsx` with React Router and per-role guards (`GuestRoute`, `ProtectedRoute`, `AdminRoute`, `SuperAdminRoute`); mobile navigation is defined in `App.js` and `AppNavigator.jsx` with tab navigation and nested stacks. The roles implemented in code are `USER`, `ADMIN` and `SUPER_ADMIN`; the `VIEWER` role documented in the specification is not implemented in the frontend.

Table 4

Role-based access matrix

Screen	Guest	USER	ADMIN	SUPER_ADMIN
Landing, guide, terms	✅	✅	✅	✅
Authentication and recovery	✅	redirect to `/dashboard`	redirect	redirect
Dashboard, environments, favorites, notifications	❌	✅	✅	✅
Profile and settings	❌	✅	✅	✅
Environment administration	❌	❌	✅	✅
Super admin panel	❌	❌	❌	✅
BLE provisioning (mobile)	❌	❌	✅	✅

Alias routes keep compatibility: `/` redirects to `/landing`, `/signup` to `/login?panel=register`, `/ranking` to `/dashboard` and `/admin/users` to `/admin/institutions`.

3. Architectural Model

3.1 Adopted architectural style

EduAirControl adopts a **microservices style with an API Gateway and event-driven communication**, implemented as a *strangler fig* migration from a legacy monolith. Each microservice corresponds to a bounded context, owns its logical database and communicates with the others through REST (synchronous) or RabbitMQ (asynchronous).

Table 5

Characteristics of the architectural style

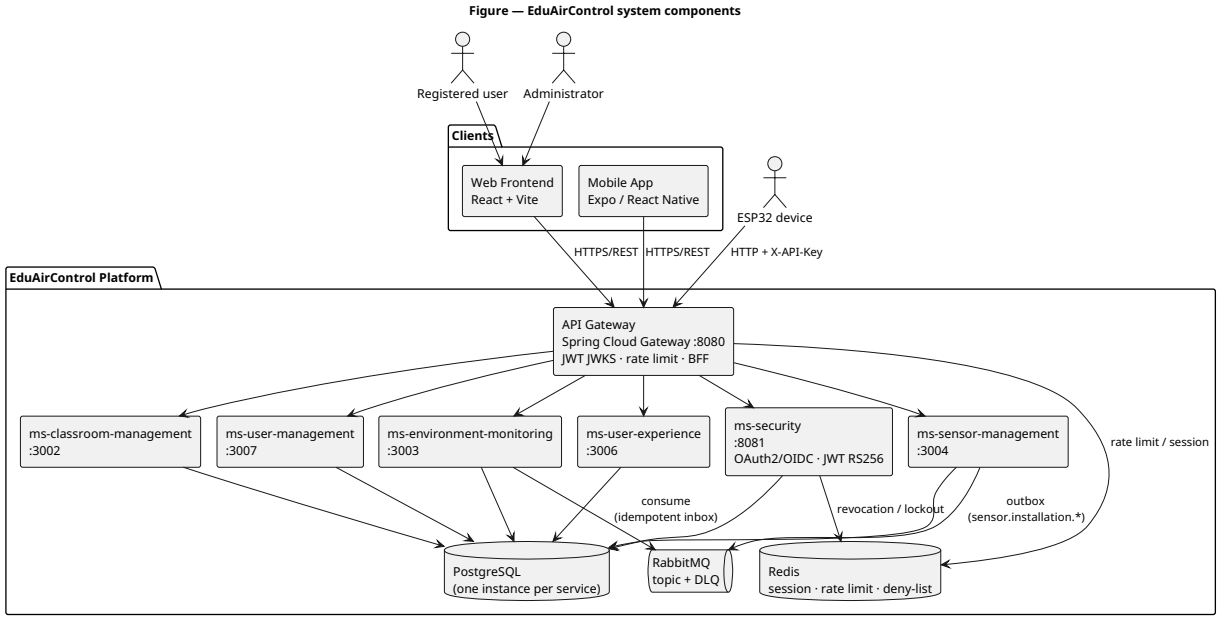
Aspect	Decision	Reference
Decomposition	One microservice per bounded context (6 services)	ADR-002
Single entry	API Gateway (Spring Cloud Gateway) with JWT validation and rate limiting	ADR-005
Synchronous communication	REST/HTTP with OpenAPI contracts	ADR-004
Asynchronous communication	RabbitMQ: topic exchange, transactional outbox, idempotent consumers	ADR-004, ADR-007
Persistence	PostgreSQL, one physical instance per service in the `infra` stack	ADR-003
Internal architecture	Hexagonal (ports and adapters) in all services	ADR-009
Security	OAuth2/OIDC + RS256 JWT with JWKS published by `ms-security`	ADR-006
Containers	Docker Compose for the full stack	ADR-010
Observability	Correlation ID at the gateway; documented stack	ADR-012

3.2 Context and container views

In the context view, the actors are the registered user, the administrator and the ESP32 devices. Web and mobile clients access exclusively through the API Gateway; IoT devices send measurements over HTTP with a per-device key (`X-API-Key`).

Figure 10

*EduAirControl system components.*Clients, API Gateway, the six microservices and infrastructure components (PostgreSQL, RabbitMQ, Redis).**



3.3 Service catalog

Table 6

Service catalog, ports and responsibilities

Service	Port	Schema	Responsibility
`api-gateway`	8080	Redis (session/ rate limit)	Routing, JWT validation (JWKS), rate limiting, BFF with httpOnly `SESSION` cookie
`ms-security`	8081	`seguridad`	Registration, authentication, OAuth2/OIDC server, RS256 JWT issuing, refresh-token rotation, revocation, institutions (tenant)
`ms-classroom-management`	3002	`classrooms`	Campuses, environment types and educational environments
`ms-sensor-management`	3004	`sensors`	Sensors, models, statuses, variables and installations; publishes events via outbox

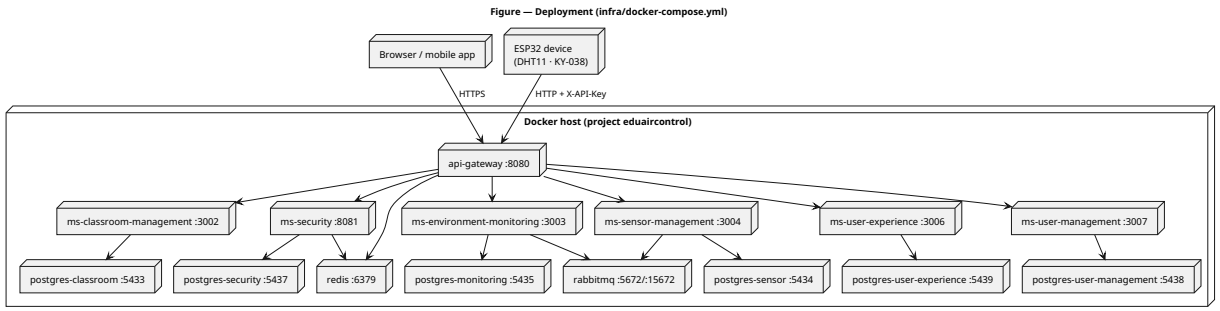
Service	Port	Schema	Responsibility
`ms-environment-monitoring`	3003	`environment_monitoring`	Measurements, thresholds, period analysis, alerts and the dashboard read model
`ms-user-management`	3007	`user_management`	User profiles
`ms-user-experience`	3006	`user_experience`	Preferences, favorites, ratings, searches and notification state

3.4 Inter-service communication

Synchronous communication uses REST over HTTP through the gateway, which propagates user identity via internal headers (`X-User-Id`, `X-User-Role`, `X-Institution-Id`, `X-Correlation-Id`). Asynchronous communication uses RabbitMQ with two topic exchanges: `eduaircontrol.environmental-data` (measurements) and `eduaircontrol.sensor` (installations). The publishing pattern is the transactional outbox and consumption uses an idempotent inbox (`ON CONFLICT DO NOTHING`), yielding at-least-once delivery without effective duplicates.

Figure 11

*Deployment diagram of the `infra/docker-compose.yml` stack.*One PostgreSQL instance per service, shared RabbitMQ and Redis.**



3.5 Security

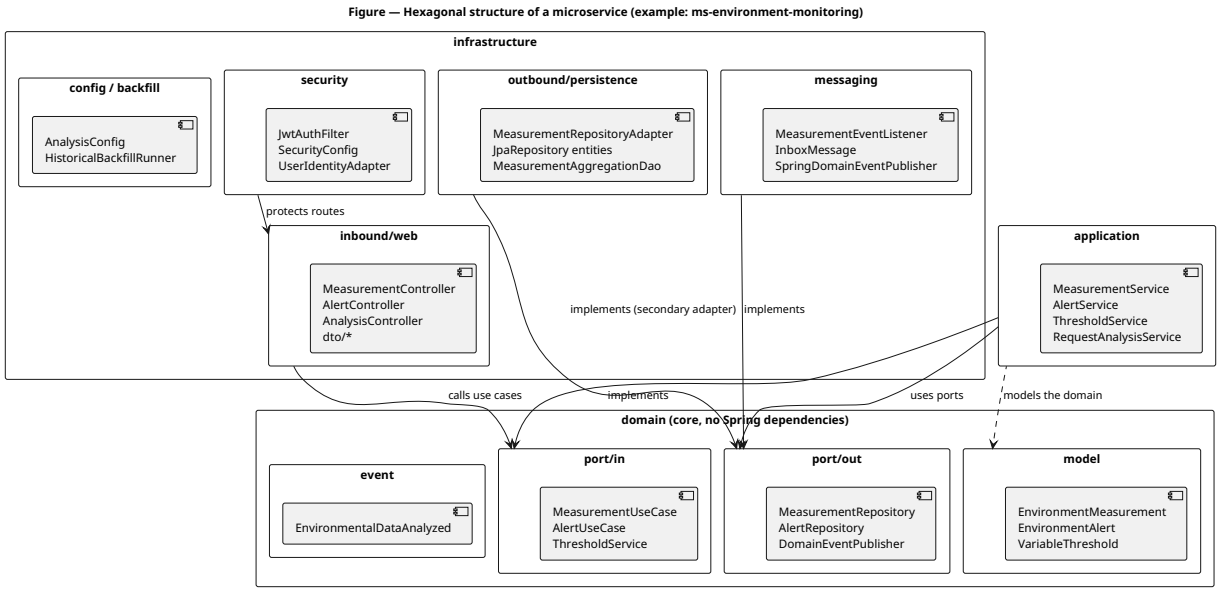
Security is designed in layers: the gateway validates signature, expiration and the token deny-list (stored in Redis) before routing; services apply fine-grained role-based authorization. Access tokens are **RS256** JWTs signed by `ms-security`, whose public key is distributed via **JWKS**; IoT devices use a per-device `X-API-Key`. The gateway BFF stores web session tokens in Redis and hands the browser only an httpOnly `SESSION` cookie, so the token never reaches JavaScript.

3.6 Internal architecture of the services

Each service follows the hexagonal architecture (ports and adapters, ADR-009): the domain defines the models and the ports (`port/in` for use cases, `port/out` for repositories and event publishing), the application layer implements the use cases and infrastructure provides the adapters (REST controllers, JPA persistence, messaging and security). The domain does not depend on Spring or JPA.

Figure 12

Hexagonal structure of `ms-environment-monitoring`.



4. Design Patterns

4.1 Architectural and microservices patterns

Table 7

Adopted patterns and their materialization in code

Pattern	Purpose	Implementation in code
Hexagonal (ports and adapters)	Decouple the domain from infrastructure	Packages `domain/port/in`, `domain/port/out`, `infrastructure/{inbound,outbound}` in all 6 services
API Gateway	Single entry, edge cross-cutting	`api-gateway` (Spring Cloud Gateway): routes, JWT, rate limit
BFF (Backend for Frontend)	Server-side session for the browser	Gateway as OAuth2 client: tokens in Redis, httpOnly `SESSION` cookie (ADR-017)
Strangler Fig	Incremental migration from the monolith	Measurement ingest still in the monolith; installations already in `ms-sensor-management`
Transactional outbox	Reliable event publishing with state changes	`outbox_event` tables in `classrooms` and `sensors`; publishing to RabbitMQ
Idempotent inbox	At-least-once consumption without duplicates	`inbox_message` in `environment_monitoring` with `ON CONFLICT DO NOTHING`
CQRS (read model)	Efficient dashboard queries	`environment_measurement`, `installation_projection`, aggregations (`MeasurementAggregationDao`)
Database per service	Data isolation per context	One schema/instance per service; no service reads another's tables (ADR-003)
Circuit breaker	Resilience against dependency failures	Documented in ADR-013; **not yet implemented** (registered drift)

Pattern	Purpose	Implementation in code
Retry with backoff	Recovery from transient failures	Work queue with retry and DLQ on the RabbitMQ consumer

4.2 GoF patterns present in the code

Table 8

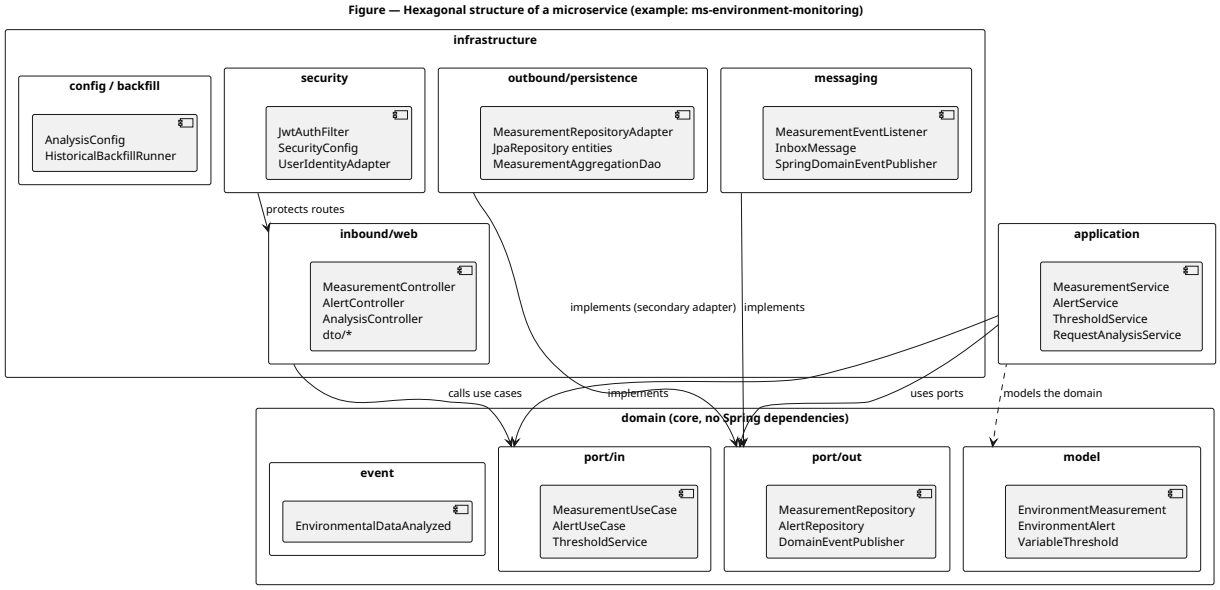
GoF patterns and their concrete use

Pattern	Category	Use in EduAirControl
Adapter	Structural	Persistence adapters (`*RepositoryAdapter`), `UserIdentityAdapter`, `ThresholdAdapter`
Repository	Structural /Data	Domain `*Repository` interfaces implemented with JPA
Builder	Creational	Domain entities with Lombok `@Builder` (`EnvironmentAlert`, `EnvironmentMeasurement`)
Factory Method	Creational	Creation of DTOs and domain entities from application services
Observer	Behavioral	Domain events (`EnvironmentalDataAnalyzed`) published to the message bus
Strategy	Behavioral	Analysis period resolution (`PeriodResolver`), threshold evaluation
Template Method	Behavioral	Common application-service flows (validate → execute → persist → publish)
Singleton	Creational	Spring configuration beans (`AnalysisConfig`, `SecurityConfig`)

4.3 Pattern traceability in the package structure

Figure 13

Traceability of the hexagonal pattern in a service's package structure.



Each primary adapter (controller) translates HTTP into a use case; each secondary adapter (persistence, messaging) implements a domain output port. This separation allows testing the domain without infrastructure and swapping adapters (for example, the event publisher) without touching business logic.

5. Data Model

5.1 Modeling principles

The data model is organized by PostgreSQL schema, one per microservice. The applied principles are: application-generated `uuid` identifiers, audit fields (`created_at`, `updated_at`, `created_by`, `updated_by`), domain separation (no service reads another's tables), referential integrity within each schema and eventual consistency across schemas through events.

5.2 Schemas and main tables

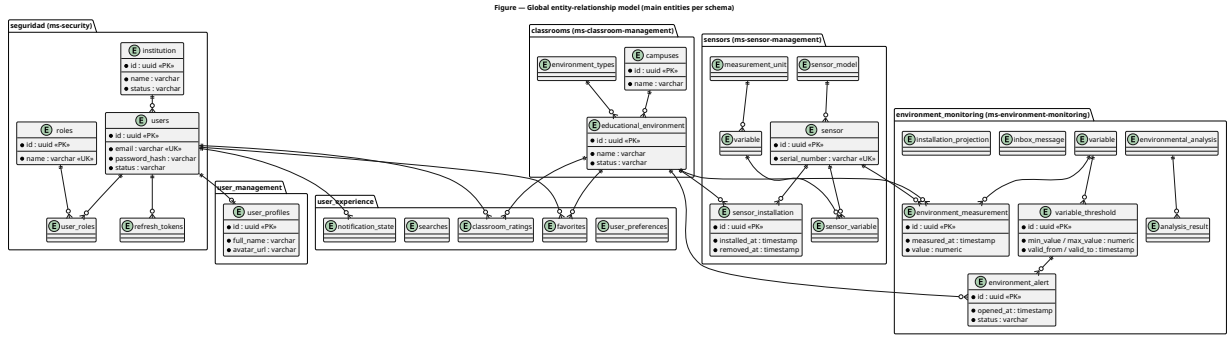
Table 9

Schemas and tables of the data model

Schema	Service	Main tables
`seguridad`	ms-security	`institution`, `users`, `user_identity`, `roles`, `user_roles`, `refresh_tokens`, `password_reset_tokens`
`classrooms`	ms-classroom-management	`campuses`, `environment_types`, `educational_environment`, `outbox_event`
`sensors`	ms-sensor-management	`sensor_model`, `sensor_status`, `measurement_unit`, `variable`, `sensor`, `sensor_installation`, `sensor_variable`, `outbox_event`
`environment_monitoring`	ms-environment-monitoring	`variable`, `educational_environment`, `variable_threshold`, `severity`, `quality_flag`, `environment_measurement`, `inbox_message`, `analysis_statuses`, `environmental_analysis`, `analysis_result`, `environment_alert`, `installation_projection`
`user_management`	ms-user-management	`user_profiles`
`user_experience`	ms-user-experience	`user_preferences`, `favorites`, `classroom_ratings`, `searches`, `notification_state`

Figure 14

Global entity-relationship model (main entities per schema).



5.3 Business rules of the model

- **Measurement is atomic**: each row of `environment_measurement` corresponds to one reading of one variable, from one sensor, in one environment, at one instant.
- **Thresholds are time-valid**: `variable_threshold` defines ranges with `valid_from`/ `valid_to`; alerts reference the evaluated threshold by foreign key, preserving the traceability of why each alert was raised.
- **Analysis is period-based**: `environmental_analysis` groups results (`analysis_result`) by time window (day, week, month, year).
- **Installations are historical**: `sensor_installation` records `installed_at`/ `removed_at`; physical deletion of a sensor with history is blocked.
- **Cross-domain identifiers are UUIDs**: `educationalEnvironmentId`, `sensorId`, `measurementUnitId` are validated as UUID at the service boundaries, without cross-existence validation (boundary documented in `ms-sensor-management`).

The complete model —data dictionary and DDL script— is delivered in `06_Base_Datos`.

6. Implementation Diagrams

The code structure of all services follows the same hexagonal skeleton, so any of them can be navigated with the same conventions. The component diagram (Figure 2) and the deployment diagram (Figure 6) complete the implementation view; the use-case, sequence and class UML diagrams are delivered in `03_Analisis_Software`.

Table 10

Code structure conventions

Layer	Package	Rule
Domain	`domain.model`, `domain.port.{in,out}`, `domain.event`, `domain.service`	No Spring or JPA dependencies
Application	`application`	Implements use cases; uses only ports
Infrastructure	`infrastructure.{inbound.web, outbound.persistence, messaging, config, backfill, security}`	Primary and secondary adapters
Shared	`shared.{security, exception, web, contract}`	Cross-cutting security, exceptions and contracts

7. Conclusions

The software design of EduAirControl consolidates four mutually consistent views, traceable to the code: a design system with real tokens shared between web and mobile and an inventory of twelve screens; a microservices architecture with an API Gateway, event-driven communication and RS256/JWKS JWT security; a set of patterns —hexagonal, repository, outbox, inbox, CQRS read model— materialized uniformly across the six services; and a per-schema data model with explicit business rules. Known drifts (documented but unimplemented circuit breaker, ADR-003 versus deployment divergence, `VIEWER` role only documented) are recorded as inputs to the quality and improvements deliverable. The design serves as a reference for continuous implementation, the test plan and the operation of the system.

References

- EduAirControl. (2026). *General project document* [Deliverable 01].
- EduAirControl. (2026). *Software requirements specification* [Deliverable 02].
- EduAirControl. (2026). *Software analysis* [Deliverable 03].
- EduAirControl. (2026). *Technical proposal* [Deliverable 04].
- EduAirControl. (2026). *Entity-relationship model, data dictionary and database script* [Deliverable 06].
- EduAirControl. (2026). *Governance documentation: system architecture, pattern guide and hexagonal architecture* (`docs-ea-control/05-architecture`).
- EduAirControl. (2026). *Domain data models* (`docs-ea-control/06-data`).
- EduAirControl. (2026). *Design system, navigation map and mockups* (`docs-ea-control/12-ux-ui`).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Richardson, C. (2018). *Microservices Patterns*. Manning Publications.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.